

# 数据结构及相关技巧

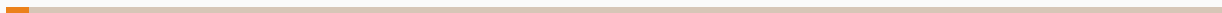
---

刘恒熙

2026 年 2 月 8 日

宁波市镇海中学

# 平衡树



平衡树通常维护一些  $(key, value)$  对（作为节点），支持快速对  $key$  在某个区间内的元素执行操作。

其中， $key$  较为关键，因为它决定了元素的顺序。

每个节点通常维护子树内  $value$  按顺序复合的结果。 $size$  一类的信息可以看成  $value$  的一部分，用相同的方式处理。

# Treap

Treap 一词来自 “tree” + “heap”。

**Tree** Treap 满足二叉搜索树的性质：

- 一个节点的左子树中的所有 key 小于它的 key，右子树的所有 key 大于它的 key。

**Heap** 每个节点被额外分配了 **(均匀独立) 随机** 权值 priority，treap 保证：

- 一个（非根）节点的 priority 小于它父亲的 priority。

这两种性质确保了 treap 的形态唯一，并且每个点的期望深度为  $\Theta(\log n)$ 。

## 分析

**深度** 设节点  $1, \dots, n$  的 key 递增, 节点  $i$  的期望深度为

$$\sum_{j=2}^i \frac{1}{j} + \sum_{j=2}^{n-i+1} \frac{1}{j} = \Theta(\log n)。$$

## 分析

**深度** 设节点  $1, \dots, n$  的 key 递增, 节点  $i$  的期望深度为

$$\sum_{j=2}^i \frac{1}{j} + \sum_{j=2}^{n-i+1} \frac{1}{j} = \Theta(\log n)。$$

**树高** 转而计算  $S = \sum_{i=1}^n 2^{\text{dep}_i}$  的期望, 其中  $\text{dep}_i$  表示节点  $i$  的深度。

- 设  $n$  个点的 treap 中  $S$  的期望为  $a_n$ 。
- $a_n = 1 + \frac{4}{n} \sum_{i=1}^{n-1} a_i$ 。
- 令  $b_n = \sum_{i=1}^n a_i$ , 那么  $b_n = b_{n-1} + a_n = \frac{n+4}{n} b_{n-1} + 1 = \left(\frac{1}{72} + o(1)\right)n^4$ 。
- 从而  $a_n = 1 + \frac{4}{n} b_{n-1} = \left(\frac{1}{18} + o(1)\right)n^3$ 。
- 因为  $2^h \leq S$ , 所以  $E(h) \leq 3 \log_2 n$  且  $P(h > 3 \log_2 n + c) \leq 2^{-c}$ 。
- 换句话说, treap 的高度通常不超过  $3 \log_2 n$ , 并且比此值大  $c$  的概率不超过  $2^{-c}$  (非常低)。
- 应用: 后缀平衡树。

## 操作: split

```
1 SPLIT( $u, k$ )
2   if  $u = \emptyset$ 
3     return  $(\emptyset, \emptyset)$ 
4   if  $u.\text{key} < k$ 
5      $v_1, v_2 \leftarrow \text{SPLIT}(u.\text{right}, k)$ 
6     return  $(\text{Node}(u.\text{data}, u.\text{left}, v_1), v_2)$ 
7   else
8      $v_1, v_2 \leftarrow \text{SPLIT}(u.\text{left}, k)$ 
9     return  $(v_1, \text{Node}(u.\text{data}, v_2, u.\text{right}))$ 
```

## 操作: merge

```
1 MERGE( $u, v$ )
2   if  $u = \emptyset \vee v = \emptyset$ 
3     | return  $u \cup v$ 
4   if  $u.\text{priority} > v.\text{priority}$ 
5     | return Node( $u.\text{data}, u.\text{left}, \text{MERGE}(u.\text{right}, v)$ )
6   else
7     | return Node( $v.\text{data}, \text{MERGE}(u, v.\text{left}), v.\text{right}$ )
```



# Treap

split 和 merge 操作足以处理大多数操作，并且与基于旋转的操作效果相同（因为 treap 的形态是唯一的）。

如果保证 treap 中没有来源相同的节点（即 priority 同时生成），treap 可以持久化。

不持久化时，上述伪代码中的 Node 直接覆盖原节点的信息；否则，Node 新建节点。

**pushdown** 标记下传在访问左右儿子前进行。

**pushup** 信息合并在新建（或更新）节点时进行。

## 操作: insert

```
1 INSERT( $u, v$ )
2   | if  $u = \emptyset$ 
3   |   | return  $v$ 
4   | if  $v.\text{priority} > u.\text{priority}$ 
5   |   | return Node( $v.\text{data}$ , SPLIT( $u, v.\text{key}$ ))
6   | if  $v.\text{key} < u.\text{key}$ 
7   |   | return Node( $u.\text{data}$ , INSERT( $u.\text{left}, v$ ),  $u.\text{right}$ )
8   | else
9   |   | return Node( $u.\text{data}$ ,  $u.\text{left}$ , INSERT( $u.\text{right}, v$ ))
```

## 操作: remove

```
1 REMOVE( $u, k$ )
2   | if  $k = u.\text{key}$ 
3   |   | return MERGE( $u.\text{left}, u.\text{right}$ )
4   | if  $k < u.\text{key}$ 
5   |   | return Node( $u.\text{data}, \text{REMOVE}(u.\text{left}, k), u.\text{right}$ )
6   | else
7   |   | return Node( $u.\text{data}, u.\text{left}, \text{REMOVE}(u.\text{right}, k)$ )
```

insert 和 remove 操作不是必需的，但是它们可以减少单点修改的常数。

# Treap 实现 key 区间有交时的合并

## 常见的简单实现

```
1 UNITE( $u, v$ )
2   if  $u = \emptyset \vee v = \emptyset$ 
3     | return  $u \cup v$ 
4   if  $u.\text{priority} < v.\text{priority}$ 
5     |  $u, v \leftarrow v, u$ 
6    $v_1, v_2 \leftarrow \text{SPLIT}(v, u.\text{key})$ 
7   return Node( $u.\text{data}, \text{UNITE}(u.\text{left}, v_1), \text{UNITE}(u.\text{right}, v_2)$ )
```

## 分析

瓶颈在 SPLIT 操作，而 SPLIT 的代价正比于它访问的节点数。

# Treap 实现 key 区间有交时的合并

假设需要合并集合  $A, B$ ，把所有元素按照 key 升序排列，分别用  $1, \dots, n$  表示这些元素及对应的节点。

计算  $\text{SPLIT}(v, u.\text{key})$  的期望贡献：

- 固定  $u$ ，不关心  $v$ 。
- 访问到  $x$ （不妨设  $u < x, u \in A, x \in B$ ）的概率是  $\frac{1}{x-u+1} \cdot \frac{1}{|[u, x] \cap B|}$ 。

因此一个来自不同集合的元素对  $(x, y)$  ( $x < y$ ) 的贡献是  $\frac{1}{y-x+1} \cdot \left( \frac{1}{|[x, y] \cap A|} + \frac{1}{|[x, y] \cap B|} \right) = \frac{1}{|[x, y] \cap A| \cdot |[x, y] \cap B|}$ 。

总和是  $\sum_{|\{i, j\} \cap A|=1, i < j} \frac{1}{|[i, j] \cap A| \cdot |[i, j] \cap B|}$ 。

设  $n_1 = |A|, n_2 = |B|$ 。有  $\sum_{|\{i, j\} \cap A|=1, i < j} \frac{1}{|[i, j] \cap A| \cdot |[i, j] \cap B|} = O\left(n_1 \log\left(\frac{n}{n_1}\right) + n_2 \log\left(\frac{n}{n_2}\right)\right)$ 。  
若  $n_1 < n_2$ ，则  $O\left(n_1 \log\left(\frac{n}{n_1}\right) + n_2 \log\left(\frac{n}{n_2}\right)\right) = O\left(n_1 \log\left(\frac{n}{n_1}\right)\right)$ 。

当来自  $A, B$  的元素几乎均匀分布时，总和取到  $\Theta\left(n_1 \log\left(\frac{n}{n_1}\right) + n_2 \log\left(\frac{n}{n_2}\right)\right)$ 。

## Treap 实现 key 区间有交时的合并

**仅 UNITE 的复杂度** 因为  $n_1 \log\left(\frac{n}{n_1}\right) + n_2 \log\left(\frac{n}{n_2}\right) = n \log n - n_1 \log n_1 - n_2 \log n_2$ , 所以合并多个总大小为  $n$  的 treap 的复杂度为  $O(n \log n)$ 。常用于树上问题。

如果有 SPLIT 操作, 那么对大小为  $2n$  的平衡树反复执行 SPLIT, UNITE, 每次 SPLIT 的时间复杂度为  $\Theta\left(\sum_{i=1}^n \sum_{j=1}^n \frac{1}{ij}\right) = \Theta(\log^2 n)$ 。

每次 UNITE 都等价于 MERGE, 而后者的时间复杂度仅为  $O(\log n)$ 。

这启发我们对 UNITE 做出修改。

# Treap 实现 key 区间有交时的合并

```
1 UNITE( $u, v$ )
2   if  $u = \emptyset \vee v = \emptyset$ 
3     | return  $u \cup v$ 
4   if  $u.\text{priority} < v.\text{priority}$ 
5     |  $u, v \leftarrow v, u$ 
6   if  $u.\text{key} < v.\text{min\_key}$ 
7     |  $v_1, v_2 \leftarrow \emptyset, v$ 
8   else if  $u.\text{key} > v.\text{max\_key}$ 
9     |  $v_1, v_2 \leftarrow v, \emptyset$ 
10  else
11    |  $v_1, v_2 \leftarrow \text{SPLIT}(v, u.\text{key})$ 
12  return Node( $u.\text{data}, \text{UNITE}(u.\text{left}, v_1), \text{UNITE}(u.\text{right}, v_2)$ )
```

# Treap 实现 key 区间有交时的合并

瓶颈包括 UNITE 本身和 SPLIT。

设  $[l, r]$  为元素全部来自同一个集合的极大区间，对每个极大区间分别计算贡献。

**UNITE 本身的贡献** 在调用 UNITE 处计算。设  $u \in [l, r]$ 。

如果在  $u$  处会调用 UNITE，那么  $u$  的 priority 一定大于  $l - 1, r + 1$  之一的 priority。

概率不超过  $\frac{1}{u-l+1} + \frac{1}{r-u+1}$ 。对  $u = l, \dots, r$  求和： $\sum_{u=l}^r \frac{1}{u-l+1} + \frac{1}{r-u+1} = \Theta(\log(r-l))$ 。

**SPLIT 的贡献** 如果在  $\text{SPLIT}(v, u.\text{key})$  时访问  $x$ （不妨设  $r < x, u \in A, x \in B$ ），那么  $u$  的 priority 是  $[l, x]$  中最大的， $x$  的 priority 是  $[l, x] \cap B$  中最大的。单个  $u$  的贡献不超过  $2 \sum_{i=1}^{+\infty} \frac{1}{(r-l+x)x} = \Theta\left(\frac{\log(r-l)}{r-l}\right)$ 。

因此一个区间的贡献是  $\Theta(\log(r-l))$ 。

所以总时间复杂度是  $\Theta(\sum (\log \text{len}_i + 1))$ ，其中  $\text{len}_i$  表示第  $i$  个极大区间的长度。

设极大区间数量（也就是元素“交替”的段数）为  $k$ ，则复杂度为  $O(k \log n)$ 。



# Treap 实现 key 区间有交时的合并

## 有 SPLIT, UNITE 操作的情况

可以证明，在只有 SPLIT, UNITE 操作时，上述“极大区间数量”  $k$  为均摊  $O(\log n)$ 。如果还有对 key 的加法（或者说平移），那么  $k$  为均摊  $O(\log V)$ ，其中  $V$  为 key 的值域大小。

因此 UNITE 的复杂度是均摊  $O(\log^2 n)$ （或者  $O(\log n \log V)$ ），比改进前少一个  $\log n$ 。

对大小为  $n = 2^{2t}$  的平衡树进行以下操作：

- 用  $2^t - 1$  次 SPLIT 操作，把平衡树分成  $2^t$  个大小为  $2^t$  平衡树  $T_0, \dots, T_{2^t-1}$ 。
- UNITE  $(T_0, T_{2^t-1}), (T_1, T_{2^t-2}), \dots, (T_{2^t-2}, T_{2^t-1})$ 。
- UNITE  $(T_0 \cup T_{2^t-1}, T_{2^t-2} \cup T_{2^t-3}), \dots, (T_{2^t-2} \cup T_{2^t-3}, T_{2^t-1} \cup T_{2^t-2})$ 。
- $\vdots$
- UNITE  $(T_0 \cup T_2 \cup \dots \cup T_{2^t-2}, T_1 \cup T_3 \cup \dots \cup T_{2^t-1})$ 。

平均每次操作  $\Theta(\log^2 n)$ ，改进前平均每次操作  $\Theta(\log^3 n)$ 。

## Treap 实现 key 区间有交时的合并

作为对比，使用动态开点线段树，平均每次操作  $\Theta(\log n)$ 。

这是因为，使用独立均匀随机的 priority 的 treap 的平衡方式，对于这种问题不合理。

Treap 无法做到优于  $\Theta(\sum(\log \text{len}_i + 1))$ ，因为需要修改的节点数量的期望是  $\Theta(\sum(\log \text{len}_i + 1))$ ，无论如何 UNITE 都无法避免这个代价。

在有对 key 的加法时，平衡树应该对 key 的值域这一维平衡。

WBLT, 即 Weight Balanced Leafy Tree。

令  $\alpha = 1 - \frac{\sqrt{2}}{2} \approx 0.292$ ,  $\beta = \frac{1}{2-\alpha} = 2 - \sqrt{2} \approx 0.586$ 。

**Leafy** 所有叶节点存储 value, 所有非叶节点恰有 2 个儿子, 存储 2 个子树的信息合并的结果。

**Weight Balanced** 对于所有非叶节点的儿子的重量 (通常定义为子树内的叶节点数) 至多是它本身的  $1 - \alpha$  倍。

**insert, remove**

与通常的二叉搜索树类似。insert 时把叶节点替换为以原节点和要插入的节点为两个儿子的非叶节点, remove 时相反。

执行完操作后, 需要对新节点到根的链上依次维护平衡。

## 维护平衡

若节点  $p = \text{Node}(x, y)$  (表示的左右儿子分别为  $x, y$  非叶节点) 不平衡, 不妨设  $y < \alpha p$  (不引起混淆时, 用节点的记号表示其重量), 即  $p$  的右儿子  $y$  太轻。

因为  $x > \frac{1-\alpha}{\alpha}y > 1$ , 设  $x = \text{Node}(z, w)$ 。

若  $w \leq \beta x$ , 把  $p$  改为  $\text{Node}(z, \text{Node}(w, y))$ 。

否则  $w > \beta x = \frac{\beta(1-\alpha)}{\alpha}y > 1$ , 设  $w = \text{Node}(u, v)$ , 把  $p$  改为  $\text{Node}(\text{Node}(z, u), \text{Node}(v, y))$ 。

Hirai 和 Yamamoto 的论文 给出了合法的  $(\alpha, \beta)$  的取值范围。

取  $(\alpha, \beta) = (\frac{1}{4}, \frac{2}{3})$  可以避免浮点数运算。

**merge**  $(\Theta(|\log(\frac{u}{v})| + 1))$

```
1 MERGE( $u, v$ )
2   if  $u = \emptyset \vee v = \emptyset$ 
3     | return  $u \cup v$ 
4   if  $\min\{u, v\} \geq \alpha(u + v)$ 
5     | return Node( $u, v$ )
6   if  $u < v$ 
7     | if  $v.\text{right} \geq \alpha(u + v)$ 
8       | return Node(MERGE( $u, v.\text{left}$ ),  $v.\text{right}$ )
9     | else
10      | return MERGE(MERGE( $u, v.\text{left}.\text{left}$ ), Node( $v.\text{left}.\text{right}, v.\text{right}$ ))
11   else
```

```
12  |   | if  $u.\text{left} \geq \alpha(u + v)$   
13  |   |   | return Node( $u.\text{left}$ ,MERGE( $u.\text{right}$ ,  $v$ ))  
14  |   | else  
15  |   |   | return MERGE(Node( $u.\text{left}$ ,  $u.\text{right}.\text{left}$ ),MERGE( $u.\text{right}.\text{right}$ ,  $v$ ))
```

这个算法在  $\alpha \in \left(0, 1 - \frac{\sqrt{2}}{2}\right]$  时正确。

**split** ( $O(\log(u))$ )

```
1 SPLIT( $u, k$ )
2   if  $k = 0$ 
3     | return ( $\emptyset, u$ )
4   if  $k = u$ 
5     | return ( $u, \emptyset$ )
6   if  $u.\text{left} < k$ 
7     |  $v_1, v_2 \leftarrow \text{SPLIT}(u.\text{right}, k - u.\text{left})$ 
8     | return ( $\text{MERGE}(u.\text{left}, v_1), v_2$ )
9   else
10    |  $v_1, v_2 \leftarrow \text{SPLIT}(u.\text{left}, k)$ 
11    | return ( $v_1, \text{MERGE}(v_2, u.\text{right})$ )
```

# 全局平衡

---



修改经典的二分猜数游戏：数被分成了若干个区间，你必须先二分出数在哪个区间，才能在一个区间内二分。

例如：有一个  $[0, 64)$  中的整数，你必须求出它在  $[0, 32)$ ,  $[32, 48)$ ,  $[48, 56)$ ,  $[56, 64)$  的哪一个，才能做出不同于区间端点的询问。

修改经典的二分猜数游戏：数被分成了若干个区间，你必须先二分出数在哪个区间，才能在一个区间内二分。

例如：有一个  $[0, 64)$  中的整数，你必须求出它在  $[0, 32)$ ,  $[32, 48)$ ,  $[48, 56)$ ,  $[56, 64)$  的哪一个，才能做出不同于区间端点的询问。

这个例子中，直接用正常的二分不会违反限制，只需要 6 次询问。而如果先对区间使用正常的二分，最坏需要 7 次。

修改经典的二分猜数游戏：数被分成了若干个区间，你必须先二分出数在哪个区间，才能在一个区间内二分。

例如：有一个  $[0, 64)$  中的整数，你必须求出它在  $[0, 32)$ ,  $[32, 48)$ ,  $[48, 56)$ ,  $[56, 64)$  的哪一个，才能做出不同于区间端点的询问。

这个例子中，直接用正常的二分不会违反限制，只需要 6 次询问。而如果先对区间使用正常的二分，最坏需要 7 次。

同理，在数据结构有嵌套时，根据外层数据结构访问内层数据结构的模式，改变内层数据结构平衡方式，可能带来优化。

# 重链剖分 + 线段树

朴素实现：单次操作的时间复杂度为  $O(\log^2 n)$ 。

把  $\Theta(\sqrt{n})$  个点的满二叉树的边替换为  $\Theta(\sqrt{n})$  个点的链，这样得到的树可以让运行时间达到  $\Theta(\log^2 n)$ 。

**全局平衡** 节点  $u$  在其重链的线段树中的权值为  $\text{light}(u) + 1$ ，其中  $\text{light}(u)$  表示  $u$  的所有轻子树的大小之和。

- 可以保证  $u$  在线段树中的深度不超过  $O\left(\log\left(\frac{\text{size}(\text{top}(u))}{\text{light}(u)+1}\right) + 1\right)$ ，其中  $\text{size}(\text{top}(u))$  是  $u$  所在重链顶端的子树大小。

单次操作的时间复杂度：

- 设在线段树上操作的节点  $u_1, u_2, \dots, u_k$  满足  $u_{i+1} = \text{parent}(\text{top}(u_i))$ ,  $\text{top}(u_k) = \text{root}$ 。
- $\sum_{i=1}^k O\left(\log\left(\frac{\text{size}(\text{top}(u_i))}{\text{light}(u_i)+1}\right) + 1\right) = O\left(\log(\text{size}(\text{top}(u_1))) + \sum_{i=2}^k \log\left(\frac{\text{size}(\text{top}(u_i))}{\text{size}(\text{top}(u_{i-1}))}\right)\right) = O(\log n)$ 。

这就是“全局平衡二叉树”。

# 点分治 + 分治

朴素实现：时间复杂度为  $O(n \log^2 n)$ 。

**全局平衡** 节点  $u$  在内层分治时的权值为  $\text{size}(u)$ ，其中  $\text{size}(u)$  表示  $u$  在分治树上的大小。

- 可以保证  $u$  在重心  $c$  处的分治深度不超过  $O\left(\log\left(\frac{\text{size}(c)}{\text{size}(u)}\right) + 1\right)$ 。

总时间复杂度为  $O(n \log n)$ 。

## 带权分治（集合）

**算法 1** 每次取权重最小的两个元素合并，新元素的权重为两个元素权重之和，直到只剩一个元素。

# 带权分治（集合）

**算法 1** 每次取权重最小的两个元素合并，新元素的权重为两个元素权重之和，直到只剩一个元素。

- 最小化  $\sum_i p_i \text{dep}_i$ ，其中  $p_i$  为元素  $i$  的权重比例， $\text{dep}_i$  为分治树中的深度。
- 权重比例为  $p$  的元素的最大深度为  $-\log_\varphi p + O(1)$ ，其中  $\varphi = \frac{\sqrt{5}+1}{2} \approx 1.618$ 。
  - $1, \varepsilon, 1 + 2\varepsilon, 1 + 2\varepsilon, 2(1 + 2\varepsilon), 3(1 + 2\varepsilon), 5(1 + 2\varepsilon), \dots$

# 带权分治（集合）

**算法 1** 每次取权重最小的两个元素合并，新元素的权重为两个元素权重之和，直到只剩一个元素。

- 最小化  $\sum_i p_i \text{dep}_i$ ，其中  $p_i$  为元素  $i$  的权重比例， $\text{dep}_i$  为分治树中的深度。
- 权重比例为  $p$  的元素的最大深度为  $-\log_\varphi p + O(1)$ ，其中  $\varphi = \frac{\sqrt{5}+1}{2} \approx 1.618$ 。
  - $1, \varepsilon, 1 + 2\varepsilon, 1 + 2\varepsilon, 2(1 + 2\varepsilon), 3(1 + 2\varepsilon), 5(1 + 2\varepsilon), \dots$

**算法 2** 令  $r_i = \lfloor \log_2 p_i \rfloor$ 。每次取  $r$  最小的两个元素  $x, y$  合并， $r_{\text{new}} = \max\{r_x, r_y\} + 1$ 。

- 权重比例为  $p$  的元素的最大深度为  $\lceil -\log_2 p \rceil$ 。
  - 没有算法能保证更优： $\frac{1}{2^k} - \varepsilon, \dots, \frac{1}{2^k} - \varepsilon, 2^k \varepsilon$ 。

因此，“点分治 + 分治”中，一个点的总分治层数的最大值是  $2 \log_2 n + O(1)$ 。利用**边分治**可以将最大值降低到  $\log_{3/2} n + O(1)$ 。



# 带权分治（序列）

**算法 1** 每次把序列分成两部分，使得两部分权重之和的差最小。

# 带权分治（序列）

**算法 1** 每次把序列分成两部分，使得两部分权重之和的差最小。

- 不能最小化  $\sum_i p_i \text{dep}_i$ 。考虑  $(10, 10, 10, 10, 29)$ 。
- 权重比例为  $p$  的元素的最大深度为  $-\log_\varphi p + O(1)$ ，其中  $\varphi = \frac{\sqrt{5}+1}{2} \approx 1.618$ 。
  - $\varepsilon, 1, \varepsilon, 1 + 3\varepsilon, 1 + 3\varepsilon, 2(1 + 3\varepsilon), 3(1 + 3\varepsilon), 5(1 + 3\varepsilon), \dots$

# 带权分治（序列）

**算法 1** 每次把序列分成两部分，使得两部分权重之和的差最小。

- 不能最小化  $\sum_i p_i \text{dep}_i$ 。考虑  $(10, 10, 10, 10, 29)$ 。
- 权重比例为  $p$  的元素的最大深度为  $-\log_\varphi p + O(1)$ ，其中  $\varphi = \frac{\sqrt{5}+1}{2} \approx 1.618$ 。
  - $\varepsilon, 1, \varepsilon, 1 + 3\varepsilon, 1 + 3\varepsilon, 2(1 + 3\varepsilon), 3(1 + 3\varepsilon), 5(1 + 3\varepsilon), \dots$

**算法 2** 令  $r_i = \lfloor \log_2 p_i \rfloor$ 。合并  $x, y$  时，新元素满足  $r_{\text{new}} = \max\{r_x, r_y\} + 1$ 。

依次添加元素  $i = 1, \dots, n$  到栈：

1. 若栈顶的两个元素  $x, y$  满足  $r_x, r_y < r_i$ ，合并  $x, y$ ，并重新添加到栈。重复执行直到条件不满足。
2. 如果栈不为空，且  $r_{\text{top}} \leq r_i$ ，将栈顶元素与  $i$  合并后添加到栈。否则，直接将  $i$  添加到栈。

最后，合并栈顶的两个元素，直到栈中只有一个元素。

- 权重比例为  $p$  ( $p < 1$ ) 的元素的最大深度为  $\lceil -\log_2 p \rceil + 1$ 。
  - 没有算法能保证更优： $\varepsilon, \frac{1}{2^k} - \varepsilon, \varepsilon, \frac{1}{2^k} - \varepsilon, \dots, \varepsilon, \frac{1}{2^k} - 2\varepsilon, \varepsilon$ 。

# 程序校验

给定  $n$  个函数  $f_1, \dots, f_n$ , 每个函数由 4 个参数  $l_i, r_i, d_i, a_i$  ( $0 \leq d_i \leq l_i \leq r_i$ ) 描述:

$$f_i(x) = \begin{cases} (x - d_i) \bmod a_i, & x \in [l_i, r_i], \\ x, & \text{otherwise.} \end{cases}$$

在线地处理  $m$  个询问:

- 给定  $x, L, R$ , 求计算  $f_R(\dots(f_{L+1}(f_L(x))))$  的过程中, 所有“生效”的取模的  $a_i$  的异或和。换句话说, 求  $\bigoplus_{i=L}^R [f_{i-1}(\dots(f_L(x))) \in [\max\{l_i, d_i + a_i\}, r_i]] \cdot a_i$ 。

时间复杂度  $O((n + m) \log V)$ , 其中  $V$  为所有  $a_i$  和所有  $x$  中的最大值。

# 程序校验

“有效”的取模至多  $O(\log V)$  次。考虑如何高效地找到下一次取模。

用持久化平衡树（这里使用 WBLT）优化 DP：对值域的维度建立平衡树，维护每个数的下一次取模以及取模前的值。

需要的操作只有分裂、复制、合并。

把 WBLT 复制  $n$  份的方法：

- 倍增，例如  $n = 19 = 16 + (2 + 1)$ ， $+$  代表 MERGE 操作。
- 仿照线段树，例如  $19 \leftarrow 10, 9 \leftarrow 5, 4 \leftarrow 3, 2 \leftarrow 2, 1 \leftarrow 1$ ，不需要 MERGE 操作，只需要简单地连接两个子树。

每次查询共  $O(\log V)$  次平衡树上查询，复杂度  $O(\log^2 V)$ 。

跳转到下一个取模操作对应的平衡树时，搜索的范围大小重新变成  $O(V)$ ，之前的信息被浪费。

## 程序校验

应该跳转到下一个平衡树**对应可能的区间的子树**。

为了使总深度至多  $O(\log V)$ ，将代表跳转到下一个子树的点的重量设为下一个子树的重量。维护平衡时不将这类节点拆开。

这会导致它们在上方  $O(1)$  层内过重，但是不影响总深度是  $O(\log V)$ 。

## 程序校验 II

给定  $n$  个操作  $(x_i, y_i, a_i, b_i)$ 。

在线地处理  $m$  个询问：

- 给定  $l, r, X$ 。
- 对  $i = l, l + 1, \dots, r$  依次执行：
  - 如果  $X \leq x_i$ ，执行  $(X, A, B) \leftarrow (X + y_i, a_i A, a_i B + b_i)$ 。
- 输出  $X, (A \bmod 2^{32}), (B \bmod 2^{32})$ 。

时间复杂度  $O((n + m) \log V)$ ，其中  $V$  为所有  $|X|$  和所有  $|\sum_{i=l}^r y_i|$  中的最大值。

## 程序校验 II

如果  $r = n$ ，只需要持久化平衡树。关键在于只考虑较新（即下标  $\leq r$ ）的标记。

如果使用持久化平衡树维护标记，那么复合标记的时间复杂度为  $O(\log n)$ 。

对标记的操作只有拼接，复合标记时，直接新建一个以两个标记为儿子的节点，时间复杂度为  $O(1)$ 。

需要加速查询。

查询时，需要求外侧平衡树的一条路径上的所有标记中，下标  $\leq r$  的标记的复合结果。

因为外侧平衡树上，靠近根的标记一定较新，所以一部分靠近根的标记可以不考虑  $\leq r$  的标记直接复合（即全局查询），只需要对一个标记考虑  $\leq r$  的限制（即前缀查询）。

表示标记的节点构成一个  $O(n \log V)$  个点的 DAG。

每个节点可达的点构成的导出子图都是树，所以 DAG 中只有  $S = O(n^2 \log^2 V)$  条路径。



## 程序校验 II

对 DAG 进行剖分。边  $e = u \rightarrow v$  为重边当且仅当经过  $e$  的路径数量分别大于经过  $u, v$  的路径数量的一半。

这样剖分后，每条路径只会经过  $O(\log S)$  条轻边。

对每条重链建立线段树，线段树按照节点走轻边后可达的节点数量平衡。

查询时需要 finger search。

总时间复杂度  $O((n + m) \log V)$ 。

## 多树二分

---

# 引入

考虑以下问题：

维护可重集  $S, T$ ，要求支持：

- 在一个集合中添加或删除元素。
- 给定  $d, k$ ，求  $\{x + d \mid x \in S\} \cup T$  中的第  $k$  小值。

# 算法 1

## 重量平衡

给定  $m$  棵重量平衡树  $T_1, \dots, T_m$ ，要求找出所有 key 中的第  $k$  小值。

维护需要考虑的所有子树，初始时空。

对于  $k = \lfloor \log_2 W \rfloor, \dots, 0$  依次执行，其中  $W$  为  $m$  棵树重量的最大值：

- 分裂（替换为两个儿子）所有重量  $\geq 2^{k+1}$  的子树，直到所有子树重量  $< 2^{k+1}$ 。
- 把所有重量  $\in [2^k, 2^{k+1})$  的树加入需要考虑的子树。
- 删除所有不需要考虑的子树。
  - 如果一个子树的 key 区间  $[l, r]$  满足：所有 key 区间  $[l', r']$  满足  $r' < x$  的子树的重量之和  $> k$ ，那么该子树的 key 都过大。反向的情形类似。
  - 对子树的 key 区间端点做 nth\_element。
    - 如果  $m$  是常数，可以直接排序。

时间复杂度  $O(\sum \log |T_i|)$ 。

# 算法 1

## 按 key 平衡

给定  $m$  棵按 key 平衡（每向上走常数层，key 的值域增大常数倍）的树  $T_1, \dots, T_m$ ，要求找出所有 key 中的第  $k$  小值。

维护需要考虑的所有子树，初始时空。

对于  $k = \lfloor \log_2 V \rfloor, \dots, 0$  依次执行，其中  $V$  为  $m$  棵树 key 值域大小的最大值：

- 分裂（替换为两个儿子）所有 key 值域大小  $\geq 2^{k+1}$  的子树，直到所有子树 key 值域大小  $< 2^{k+1}$ 。
- 把所有 key 值域大小  $\in [2^k, 2^{k+1})$  的树加入需要考虑的子树。
- 删除所有不需要考虑的子树。

时间复杂度  $O(m \log V)$ 。

值得注意的是，有时候可以用线段树代替平衡树。

## 算法 2

给定  $m$  棵重量平衡树  $T_1, \dots, T_m$ ，要求找出所有 key 中的第  $k$  小值。

设  $S = \sum_{i=1}^m |T_i|$ 。

以下仅考虑二分的一步。

不妨设  $k < \frac{1}{2}S$ 。需要在保证不删除前一半元素的情况下，删除一定比例的元素。

令  $\alpha = 0.69, \beta = 0.74$ 。

首先对每棵平衡树  $T_i$  找出任意一个排名的比例在  $[\alpha, \beta]$  中的元素  $k_i$ 。

单次操作时间复杂度  $O(\log(\frac{1}{b-a})) = O(1)$ 。

在  $O(m)$  的时间内，将平衡树按照  $k_i$  重排为  $(T_1, \dots, T_{p-1}), T_p, (T_{p+1}, \dots, T_m)$ ，使得：

- 重排后的  $k_i$  满足  $k_1, \dots, k_{p-1} < k_p < k_{p+1}, \dots, k_m$ 。
- $\sum_{i=1}^p |T_i| \geq \frac{1}{2\alpha} (\approx 0.72) S$ 。
- $\sum_{i=p}^m |T_i| \geq (1 - \frac{1}{2\alpha}) (\approx 0.28) S$ 。

## 算法 2

小于  $k_p$  的元素占比至少为  $\alpha \cdot \frac{1}{2\alpha} = \frac{1}{2}$ ，所以大于  $k_p$  的元素可以删除。

对于  $i = p, \dots, m$ ，删除  $T_i$  中大于  $k_i$  的元素。持久化，单次操作  $O(1)$ 。

这些元素的占比至少为  $(1 - \beta)(1 - \frac{1}{2\alpha}) \approx 0.072$ 。

问题规模缩小了常数倍。

总时间复杂度  $O(m \log S)$ 。

可以优化到  $O(\sum \log |T_i|)$ 。

## (Ynoi 2011) ODT

给定一棵  $n$  个点的树，点有点权。

处理  $m$  个操作，每个操作为以下两种之一：

- 给定  $x, y, z$ ，将节点  $x$  到  $y$  的简单路径上的所有点的点权加  $z$ 。
- 给定  $x, y$ ，求与  $x$  的距离（简单路径的边数）小于等于  $y$  的所有点中，点权的最小值。

时间复杂度  $O(m \log n)$ 。



# 莫队算法与等价类分治

---

# 莫队算法

莫队算法通常用于处理这类（离线）问题：

- 数据结构在状态  $(l, r)$  时能求出询问  $(l, r)$  的答案。
- 能快速从状态  $(l, r)$  转移到  $(l \pm 1, r), (l, r \pm 1)$ 。

取参数  $B = \Theta\left(\frac{n}{\sqrt{q}}\right)$ 。

将所有询问  $(l, r)$  按照  $\left(\lfloor \frac{l}{B} \rfloor, (-1)^{\lfloor \frac{l}{B} \rfloor} r\right)$  排序。

依次处理所有询问，端点移动次数总和不超过  $O(n\sqrt{q})$ 。

# 莫队算法

## 常数下界

给定序列长度  $n$  和询问次数  $q$ , 满足  $q = \omega(1) \cap o(n^2)$ 。

取参数  $B = \left\lceil \frac{n}{2\sqrt{q}} \right\rceil$ 。

对于所有  $1 \leq i \leq j \leq \frac{n}{B}, 2 \mid (j - i)$ , 添加询问  $(i \cdot B, j \cdot B)$ 。

这样会产生  $(1 + o(1))\frac{1}{4}\left(\frac{n}{B}\right)^2 = (1 + o(1))q$  个询问, 且两两之间的曼哈顿距离  $\geq 2B$ 。

因此处理这些询问至少需要移动端点  $(1 + o(1))q \cdot 2B = (1 + o(1))n\sqrt{q}$  次。

添加或删除  $o(q)$  个询问使得询问个数恰为  $q$ 。

## 常数上界

给定序列长度  $n$  和询问次数  $q$ , 满足  $q = \omega(1) \cap o(n^2)$ 。

取参数  $B = \left\lceil \frac{n}{\sqrt{q}} \right\rceil$ 。对  $i = 0, \dots, B-1$ , 令  $a_i = \sum_{(l,r) \in Q} \left| ((l-i) \bmod B) - \frac{B}{2} \right|$ 。

每个询问对  $\sum_{i=0}^{B-1} a_i$  的贡献为  $\sum_{i=0}^{B-1} \left| i - \frac{B}{2} \right| = \left( \frac{1}{4} + o(1) \right) B^2$ 。

设  $a_h = \min_{i \in [0, B-1]} a_i$ , 则  $a_h \leq \left( \frac{1}{4} + o(1) \right) qB$ 。

将所有询问  $(l, r)$  按照  $\left( \left\lfloor \frac{l-h}{B} \right\rfloor, (-1)^{\left\lfloor \frac{l-h}{B} \right\rfloor} r \right)$  排序。

设排序后的询问为  $(l_1, r_1), (l_2, r_2), \dots, (l_q, r_q)$ 。

$$\sum_{i=1}^{q-1} |l_{i+1} - l_i| \leq \sum_{i=1}^{q-1} \left( \left| ((l_i - h) \bmod B) - \frac{B}{2} \right| + \left| ((l_{i+1} - h) \bmod B) - \frac{B}{2} \right| \right) + \left\lfloor \frac{n}{B} \right\rfloor \cdot B \leq 2a_h + n = \left( \frac{1}{2} + o(1) \right) qB = \left( \frac{1}{2} + o(1) \right) n\sqrt{q}。$$

$$\sum_{i=1}^{q-1} |r_{i+1} - r_i| \leq \sum_{i=-1}^{\left\lfloor \frac{n-h}{B} \right\rfloor} (n - (iB + h)) = \left( \frac{1}{2} + o(1) \right) \frac{n^2}{B} = \left( \frac{1}{2} + o(1) \right) n\sqrt{q}。$$

总移动次数不超过  $(1 + o(1))n\sqrt{q}$ 。

# 等价类分治

**问题** 给定  $n$  个元素  $a_1, \dots, a_n$ ，处理  $q$  个询问：

- 给定  $S_i \in \mathcal{S}$ ，求  $\prod_{j \in S_i} a_j$ 。

**解法** 选定合适的参数  $B$ ，以  $B$  为最大块长对  $a_1, \dots, a_n$  分块。对于一个块  $(a_l, \dots, a_r)$ ， $(r - l + 1 \leq B)$ ：

- 需要对所有  $T \in \{S \cap \{l, \dots, r\} \mid S \in \mathcal{S}\}$  计算  $\prod_{i \in T} a_i$ 。
- 使用分治，先对  $(a_l, \dots, a_{\text{mid}}), (a_{\text{mid}+1}, a_r)$  计算。
- 对于所有  $T$ ，令  $T_1 = \{i \leq \text{mid} \mid i \in T\}, T_2 = \{i > \text{mid} \mid i \in T\}$ 。
- 那么  $\prod_{i \in T} a_i = \left( \prod_{i \in T_1} a_i \right) \left( \prod_{i \in T_2} a_i \right)$ ，
- 且  $\prod_{i \in T_1} a_i, \prod_{i \in T_2} a_i$  已经在递归调用中计算。
- 令  $f(m) = \max_{r-l+1=m} |\{S \cap \{l, \dots, r\} \mid S \in \mathcal{S}\}|$ 。
- 时间复杂度为  $O(\frac{n}{B}(q + T(B)))$ ，其中  $T(m) = 2T(\frac{m}{2}) + O(f(m))$ 。

提示：如果莫队 + 线段树不能突破  $\Theta(n\sqrt{q} \log n)$ ，不妨试试等价类分治。

## 实例

- 同时给定排列  $p_1, \dots, p_n$ , 定义  $\mathcal{S} = \{\{i \mid p_i \in [l, r]\} \mid 1 \leq l \leq r \leq n\}$ .
  - $f(m) = O(m^2), T(m) = O(m^2)$ . 令  $B = \Theta(\sqrt{q})$ , 时间复杂度  $O(n\sqrt{q})$ .
- 询问没有特殊性质, 即  $\mathcal{S} = 2^{\{1, \dots, n\}}$ .
  - $f(m) = O(2^m), T(m) = O(2^m)$ . 令  $B = \max\{\lfloor \log_2 q \rfloor, 1\}$ , 时间复杂度  $O\left(\frac{nq}{\log q}\right)$ .
  - 即 Method of Four Russians.
- 同时给定排列  $p_1, \dots, p_n$ , 定义  $\mathcal{S} = \{\{i \mid p_i \leq r\} \mid 1 \leq r \leq n\}$ .
  - $f(m) = O(m), T(m) = O(m \log m)$ . 令  $B = \Theta\left(\frac{q}{\log q}\right)$ , 时间复杂度  $O(n \log q)$ .
  - 等价于先以  $B$  为块长分块, 然后对每块建立线段树。

# Epilogue of Happiness

给定一棵  $n$  个点的树，节点 1 是根，点有点权。

给定长度为  $m$  的节点序列  $o_1, \dots, o_m$ 。

处理  $q$  次询问（允许离线）：

- 给定  $l, r, x$ ，求节点  $x$  到根的路径上，所有满足以下条件的点  $u$  的点权之和：
  - 节点  $o_l, \dots, o_r$  中，有奇数个节点在  $u$  的子树内（重复出现计入多次）。

时间复杂度  $O(n\sqrt{q})$ 。

## 一些常数优化

---



# 关于内存访问

原则：一起访问的信息尽量连续。

数据结构中，一个节点的信息用 struct 存储，而不是在多个数组中存储。

树状数组的下标访问  $a[i]$  改为  $a[i+(i>>10)]$ 。在树状数组较大时有效。

需要二分的数组采用 Eytzinger Layout：元素  $i$  ( $i = 1, \dots, 2^k - 1$ ) 存储在  $a[(i+(1<<k))>>(\_\_builtin\_ctz(i)+1)]$ 。

对于动态开点的数据结构，把信息存在父亲处。具体效果未知，可能在处理区间修改时更有效。

# 关于运算

如果懒标记有交换律，可以不下传懒标记。如果还可逆，那么可以把区间修改拆成两个单点修改，这样不需要懒标记。

- 有交换律：pushup 改为  $\text{value} \leftarrow \text{tag} \times (\text{left.value} + \text{right.value})$ 。
- 有交换律，可逆：改为维护  $(\text{value}, \text{tag})$ ， $(a, b) + (c, d) = (a + b \times c, b \times d)$ 。

设计更好的信息，例如 [NOIP2022] 比赛。

$a = (a + (\text{ull})b * c + (\text{ull})(\text{MOD} - d) * e) \% \text{MOD}$ 。

# 追求更优的结构

---

## 【北大集训 2021】末日魔法少女计划

对于给定的  $n, k$ ，你需要构造一个只含 0, 1 的矩阵  $A_{i,j}$ ， $0 \leq i, j \leq n$ ，满足：

1.  $A_{i,i} = 1$ 。
2.  $A_{i,i+1} = 1$ 。
3. 对  $i > j$  有  $A_{i,j} = 0$ 。
4. 若  $A_{i,j} = 1, j - i > 1$ ，则存在  $i > t > j$ ，满足  $A_{i,t} = A_{t,j} = 1$ 。
5. 对  $i < j$  有  $(A^k)_{i,j} > 0$ 。

你需要输出满足  $A_{i,j} = 1$  且  $j - i > 1$  的每个  $(i, j)$ ，设这样的  $(i, j)$  共有  $m$  个。

根据  $m$  进行评分，需要使  $m$  较小。

## 【北大集训 2021】末日魔法少女计划

对树形结构进行 DP 即可通过此题。

当  $k = 2$  时,  $m = O(n \log n)$ 。

当  $k = 3$  时,  $m = O(n \log \log n)$ 。

当  $k = 4$  时,  $m = O(n \log^* n)$ 。

存在  $k = O(\alpha(n))$ , 使得  $m = O(n\alpha(n))$ 。

# 致谢

感谢李欣隆专家与我交流。

感谢大家的倾听。